

SOLUTION SHEET

Invigilator's Signature: _____



Indian Institute of Technology Bhubaneswar
Department of Computer Science and Engineering
School of Electrical and Computer Sciences
Deep Learning (CS6L038)

Date: May 3, 2025

Spring End-Semester Examination (2025)

Time: 3 hours

Maximum Marks: 40

Student Details

Name: _____ Roll No: _____
School/Dept.: _____ Degree: _____ Semester: _____

To be filled by the Evaluator

Question:	1	2	3	4	Total
Points:	2	3	10	25	40
Score:					

Instructions

1. Ensure that you have seated yourself according to the examination seating plan.
 2. Do not carry a mobile phone or any similar device, even if it is switched off. Loose papers, notes, or books must not be in your possession, regardless of their relevance to the exam.
 3. Write your answers in the space provided in this booklet using **PEN ONLY**. Do not overwrite and strikethrough the answers. Do not collect separate answer sheets.
 4. Be precise and answer to the point. "Keep it short, and simple".
 5. Avoid tearing off any pages. If the answer script is damaged, report it immediately to the invigilator.
 6. No questions during the examination should be entertained. If you have any doubt which is not clear from the the information given in the question, write your assumption clearly.
 7. Always present your admit card or identity card when requested by the invigilator. It is your responsibility to ensure that your attendance is properly recorded.
 8. You may leave the examination hall for washroom breaks or to drink water. Record your absence in the register provided.
 9. Do not leave your seat after the exam until the invigilator has collected your answer script.
 10. Any attempt to gather information during the examination, either inside or outside the hall, will be considered an 'unfair means'. This includes exchanging information or helping others to do so. Avoid engaging in any improper behavior.
 11. If space is not enough for one question, you can write the rest of the answer on the rough pages.
- Note:** This question paper contains sixteen (16) pages (including this instruction page and rough pages at the back). All questions are compulsory to answer. Use the space provided in the sheet to write your answers. Separate answer sheets won't be provided. Space for rough work is provided at the end of the question paper.

Violation of any of the instructions may lead to disciplinary action.

*** Wish you all the best ***

Signature of the Student

Please go on to the next page...

1. For each of the following statements, indicate **TRUE** / **FALSE**.

For the FALSE ones, write a corrected statement in a reasonable manner. **Full marks would be awarded only if the reasoning is correct.** (0.5 × 4 = 2)

- (a) Increasing the number of convolutional filters in a CNN layer always improves the model's ability to generalize to unseen data.

Answer: FALSE, More filters increase the model's capacity, but if data is limited, the model may learn unnecessary detail.

- (b) In a CNN, padding is used to ensure that the spatial dimensions of the output feature maps do not shrink after each convolutional operation.

Answer: TRUE

- (c) Theoretically, a vanilla RNN with sufficient hidden units and training time can always model long-term dependencies in sequential data.

Answer: TRUE

- (d) In gradient descent, using a higher learning rate guarantees faster convergence to the global minimum for all types of optimization problems.

Answer: FALSE, A higher learning rate may speed-up convergence initially, but can overshoot as the training process proceeds further.

2. Answer the following questions.

(1 × 3 = 3)

- (a) Write the weight update formula in case of RMSProp optimization?

Answer:

$$\gamma_t = \beta \gamma_{t-1} + (1 - \beta) g_t \odot g_t$$

$$\theta_{t+1} = \theta_t - \frac{\gamma}{\beta + \sqrt{\gamma_t}} \odot g_t$$

- (b) What are the advantages of CNN over MLP?

Answer:

- ① Locality
- ② Invariance to translation
- ③ Hierarchical compositionality

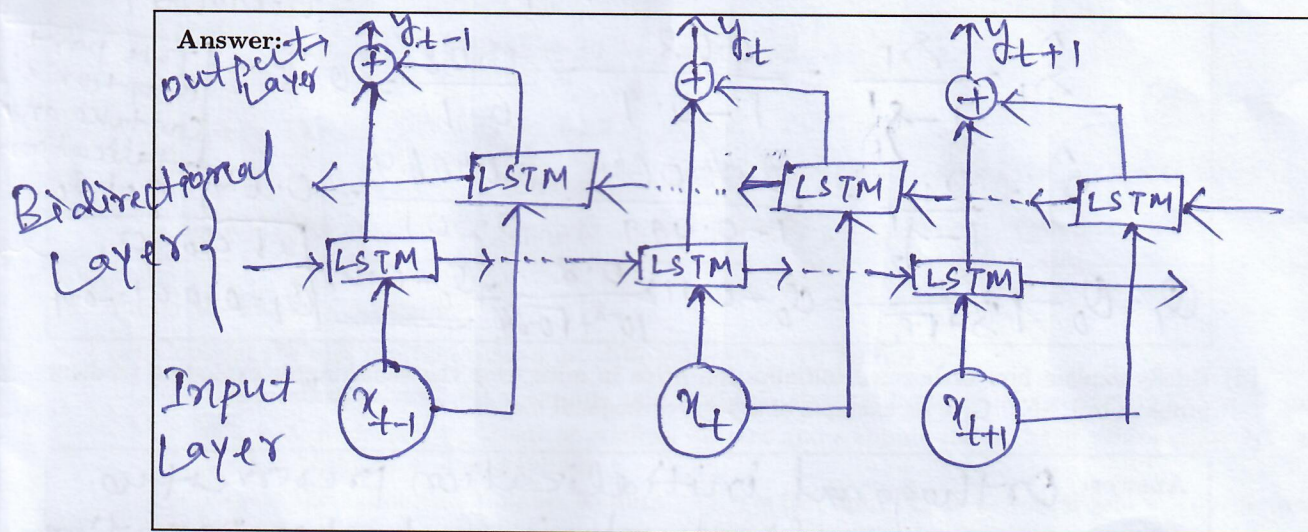
- (c) In an LSTM network, what could be a potential disadvantage of using only the final hidden state for classification tasks involving long sequences? Justify your answer in one sentence.

Answer: It may lose important information from earlier time steps, as the final hidden state might not retain all relevant context from a long sequence.

3. Answer the following questions.

(2 × 5 = 10)

(a) Draw the schematic diagram of a bidirectional LSTM module.



(b) A fully connected layer in a neural network has 128 input neurons and 64 output neurons. Using Xavier initialisation for a $\tanh$ activation function, compute the range $[-a, a]$ from which the weights should be drawn if using a uniform distribution. What will be the range in case of He initialization?

Answer: (a) Xavier: $a = \sqrt{\frac{6}{n_{in} + n_{out}}} = \sqrt{\frac{6}{128 + 64}} = \sqrt{\frac{6}{192}} \approx 0.176$

So, weights are drawn from $[-0.176, 0.176]$

(b) He: $a = \sqrt{\frac{6}{n_{in}}} = \sqrt{\frac{6}{128}} \approx 0.216$

So, weights are drawn from $[-0.216, 0.216]$

- (c) A parameter θ in a neural network is being optimised using the Adam optimizer. At time step $t = 1$, the following values are given (use the notations as discussed during the theory classes):

At $t=1$: $g_1 = 0.8$, $\gamma = 0.01$, $\rho_1 = 0.9$, $\rho_2 = 0.999$, $\delta = 10^{-8}$, $s_0 = 0$, $r_0 = 0$

Compute the updated parameter value. Give your answer correct up to 4 decimal places.

Answer:

$$\hat{s}_1 = \rho_1 s_0 + (1 - \rho_1) g_1 = 0.9 \times 0 + 0.1 \times 0.8 = 0.08$$

$$\hat{r}_1 = \rho_2 r_0 + (1 - \rho_2) g_1 g_1 = 0.999 \times 0 + 0.001 \times 0.8 \times 0.8 = 0.00064$$

$$\hat{s}_1 = \frac{s_1}{1 - \rho_1} = \frac{0.08}{1 - 0.9} = \frac{0.08}{0.1} = 0.8$$

$$\hat{r}_1 = \frac{r_1}{1 - \rho_2} = \frac{0.00064}{1 - 0.999} = \frac{0.00064}{0.001} = 0.64$$

Two part is optional, can take any initialization for s_0

$$\theta_1 = \theta_0 - \gamma \frac{\hat{s}_1}{\delta + \sqrt{\hat{r}_1}} = \theta_0 - 0.01 \times \frac{0.8}{10^{-8} + \sqrt{0.64}} \approx \theta_0 - 0.01$$

if $\theta_0 = 0$, $\theta_1 = 0 - 0.01 = -0.01$

- (d) Briefly explain how orthogonal initialization helps in mitigating the vanishing or exploding gradient problem in RNNs. Give an example of a 3×3 orthogonal matrix.

Answer: Orthogonal initialization preserve the norm of the gradients during backpropagation in RNNs by maintaining the eigen values of the weight matrix close to 1, preventing vanishing or exploding gradients over long sequences.

3x3 orthogonal matrix:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}$$

(put any value of θ)

- (e) A grayscale image of size 32×32 is passed through the following CNN layers:

- A convolutional layer with 10 filters, each of size 5×5 , stride = 1, and no padding
- Followed by a max pooling layer of size 2×2 , with stride = 2

Now suppose we change the stride of the convolutional layer to 2 and add padding of size 2.

- Compute the output size (width $\times$ height) of the feature map before and after the change in stride and padding.
- Based on your results, explain briefly (in one sentence) how changing stride and padding affects the spatial dimensions and computation.

Answer: (i) Before stride, input = 32×32 , filter: 5×5 , $p=0$, $s=1$.

$$\text{output size} = \left\lfloor \frac{32-5}{1} \right\rfloor + 1 = 28$$

After max pooling (2×2 , $s=2$), size = $\frac{28}{2} = 14$.

$$\therefore \text{output size} = 14 \times 14 \text{ (Before stride)}$$

(ii) After stride, input: 32×32 , filter: 5×5 , $p=2$, $s=2$.

$$\text{output size} = \left\lfloor \frac{32-5+2 \times 2}{2} \right\rfloor + 1 = \left\lfloor \frac{31}{2} \right\rfloor + 1 = 15 + 1 = 16$$

After max pooling (2×2 , $s=2$), size = $\frac{16}{2} = 8 \rightarrow (8 \times 8)$ Answer.

(iii) Increasing stride and adding padding reduces the spatial dimensions more aggressively, lowering computation and memory requirements.

4. Answer the following questions.

(5 × 5 = 25)

(a) Consider the cost function defined as: $J(\theta_1, \theta_2) = (\theta_1 - 3)^2 + (\theta_2 + 2)^2$.

- Use gradient descent with learning rate $\eta = 0.1$, starting from initial parameters $\theta_1^{(0)} = 0$ and $\theta_2^{(0)} = 0$. Perform 4 iterations of gradient descent and compute the updated values of θ_1 , θ_2 , and the cost $J(\theta_1, \theta_2)$ at each iteration.
- Repeat the above with a higher learning rate $\eta = 0.8$, using the same initial values. Observe and explain the difference in the convergence pattern for the two learning rates.

Answer: Given cost function $J(\theta_1, \theta_2) = (\theta_1 - 3)^2 + (\theta_2 + 2)^2$

Gradient - $\frac{dJ}{d\theta_1} = 2(\theta_1 - 3)$, $\frac{dJ}{d\theta_2} = 2(\theta_2 + 2)$

$$\theta_{t+1} = \theta_t - \eta \frac{dJ}{d\theta}$$

Case-1 ($\eta = 0.1$), $\theta_1^{(0)} = 0$ and $\theta_2^{(0)} = 0$.

Iteration	θ_1	θ_2	ΔJ	updated θ_1, θ_2	Cost J
0	0	0	(-6, 4)	(0.6, -0.4)	8.32
1	0.6	-0.4	(-4.8, 3.2)	(1.08, -0.72)	5.3248
2	1.08	-0.72	(-3.84, 2.56)	(1.464, -0.976)	3.41
3	1.464	-0.976	(-3.072, 2.048)	(1.7712, -1.1808)	2.187

Put the values of θ_1 and θ_2 in $J(\theta_1, \theta_2)$ to get the cost

Question 4 continues on the next page...

Answer:

Case-2 $\eta = 0.8$

Iter				updated		Cost J
	θ_1	θ_2	ΔJ	θ_1	θ_2	
0	0	0	$(-6, 4)$	4.8	-3.2	4.68
1	4.8	-3.2	$(3.6, -2.4)$	1.92	-1.28	1.6848
2	1.92	-1.28	$(-2.16, 1.44)$	3.648	-2.432	0.6065
3	3.648	-2.432	$(1.296, -0.864)$	2.6088	-1.7408	0.22

Observations

Higher learning rate ($\eta = 0.8$) converges much faster due to large steps, but may risk instability in non-convex functions. However, lower learning rate ($\eta = 0.1$) converges slowly and in a safe manner. Hence, a slow learning rate is always preferred. Adaptive learning rate scheduler is also a good option.

- (b) Consider a time series where each input at time step t ($t \in \{1, 2, 3, 4\}$) is a 2-dimensional vector:

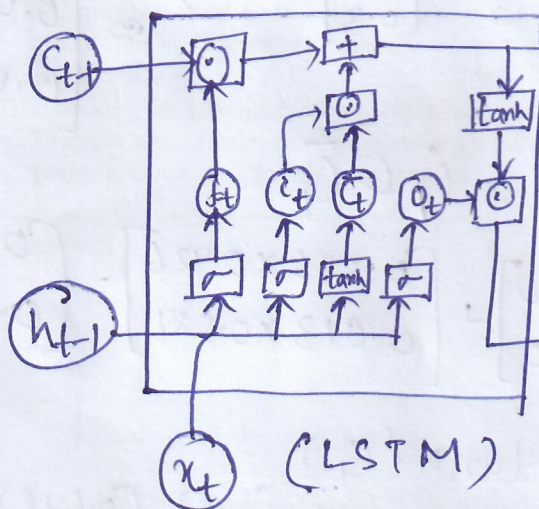
$$x = \begin{bmatrix} 0.2 \\ 0.3 \end{bmatrix}, \begin{bmatrix} 0.4 \\ 0.5 \end{bmatrix}, \begin{bmatrix} 0.6 \\ 0.7 \end{bmatrix}, \begin{bmatrix} 0.8 \\ 0.9 \end{bmatrix}, \text{ Initial states: } h_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, c_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

You are tasked with building a single-layer LSTM model to predict the next value in the sequence. Draw the schematic diagram of the LSTM module. The LSTM uses the following initial weight matrices and bias vectors. The notations are same as discussed during the theory classes.

$$W_f = \begin{bmatrix} 0.5 & 0.3 \\ 0.1 & 0.6 \\ 0.4 & 0.2 \\ 0.2 & 0.4 \end{bmatrix}, b_f = \begin{bmatrix} 0.1 \\ 0.2 \end{bmatrix}, W_i = \begin{bmatrix} 0.4 & 0.6 \\ 0.2 & 0.7 \\ 0.5 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}, b_i = \begin{bmatrix} 0.2 \\ 0.3 \end{bmatrix}, W_c = \begin{bmatrix} 0.3 & 0.5 \\ 0.3 & 0.6 \\ 0.2 & 0.3 \\ 0.4 & 0.7 \end{bmatrix}, b_c = \begin{bmatrix} 0.3 \\ 0.4 \end{bmatrix}, W_o = \begin{bmatrix} 0.6 & 0.4 \\ 0.1 & 0.6 \\ 0.3 & 0.1 \\ 0.5 & 0.3 \end{bmatrix}, b_o = \begin{bmatrix} 0.4 \\ 0.1 \end{bmatrix}$$

For the first time step $t = 1$, compute the new cell state and hidden state. Round all intermediate results to **3 decimal places**. Briefly explain how the LSTM model can capture temporal dependencies better than a simple RNN.

Answer:



$$\begin{aligned} f_t &= \sigma(W_f^T[h_{t-1}, x_t] + b_f) \\ i_t &= \sigma(W_i^T[h_{t-1}, x_t] + b_i) \\ o_t &= \sigma(W_o^T[h_{t-1}, x_t] + b_o) \\ \tilde{c}_t &= \tanh(W_c^T[h_{t-1}, x_t] + b_c) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\ h_t &= o_t \odot \tanh(c_t) \end{aligned}$$

$$x_1 = \begin{bmatrix} 0.2 \\ 0.3 \end{bmatrix} \quad h_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad c_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$[h_0, x_1] = [0 \quad 0 \quad 0.2 \quad 0.3]^T$$

Forget gate

$$f_1 = \sigma \left(\begin{bmatrix} 0.5 & 0.1 & 0.4 & 0.2 \\ 0.3 & 0.6 & 0.2 & 0.4 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0.2 \\ 0.3 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.2 \end{bmatrix} \right) = \sigma \left(\begin{bmatrix} 0.29 \\ 0.36 \end{bmatrix} \right) \approx \begin{bmatrix} 0.560 \\ 0.589 \end{bmatrix}$$

Input gate

$$i_1 = \sigma \left(\begin{bmatrix} 0.4 & 0.2 & 0.5 & 0.3 \\ 0.6 & 0.7 & 0.2 & 0.4 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0.2 \\ 0.3 \end{bmatrix} + \begin{bmatrix} 0.2 \\ 0.3 \end{bmatrix} \right) = \sigma \left(\begin{bmatrix} 0.39 \\ 0.46 \end{bmatrix} \right) \approx \begin{bmatrix} 0.596 \\ 0.613 \end{bmatrix}$$

Answer:

output gate $\rightarrow O_t = \sigma \left(\begin{bmatrix} 0.6 & 0.1 & 0.3 & 0.5 \\ 0.4 & 0.6 & 0.1 & 0.3 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0.2 \\ 0.3 \end{bmatrix} + \begin{bmatrix} 0.4 \\ 0.1 \end{bmatrix} \right) = \sigma \left(\begin{bmatrix} 0.61 \\ 0.21 \end{bmatrix} \right) \approx \begin{bmatrix} 0.648 \\ 0.552 \end{bmatrix}$

proposal cell state $\rightarrow \bar{C}_t = \tanh \left(\begin{bmatrix} 0.3 & 0.3 & 0.2 & 0.4 \\ 0.5 & 0.6 & 0.3 & 0.7 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0.2 \\ 0.3 \end{bmatrix} + \begin{bmatrix} 0.3 \\ 0.4 \end{bmatrix} \right) = \tanh \left(\begin{bmatrix} 0.46 \\ 0.67 \end{bmatrix} \right) \approx \begin{bmatrix} 0.426 \\ 0.581 \end{bmatrix}$

cell state $\rightarrow C_t = f_1 \odot C_0 + \bar{f}_1 \odot \bar{C}_t$
 $= \begin{bmatrix} 0.560 \times 0 \\ 0.589 \times 0 \end{bmatrix} + \begin{bmatrix} 0.596 \times 0.426 \\ 0.613 \times 0.581 \end{bmatrix} = \begin{bmatrix} 0.254 \\ 0.356 \end{bmatrix}$

Hidden state $\rightarrow h_t = O_t \odot \tanh(C_t)$
 $= \begin{bmatrix} 0.648 \times \tanh(0.254) \\ 0.552 \times \tanh(0.356) \end{bmatrix} = \begin{bmatrix} 0.648 \times 0.249 \\ 0.552 \times 0.343 \end{bmatrix}$

$\Rightarrow h_t \approx \begin{bmatrix} 0.161 \\ 0.189 \end{bmatrix} \quad C_t = \begin{bmatrix} 0.254 \\ 0.356 \end{bmatrix}$

$\rightarrow$ Simple RNNs suffer from vanishing gradients, where gradients shrink during back propagation, limiting long-term dependency learning. LSTM addresses this issue via:

- (i) cell state: keeps info across many time steps
- (ii) forget gate: selectively forgets or discards past data.
- (iii) Input gate: controls new information integration.

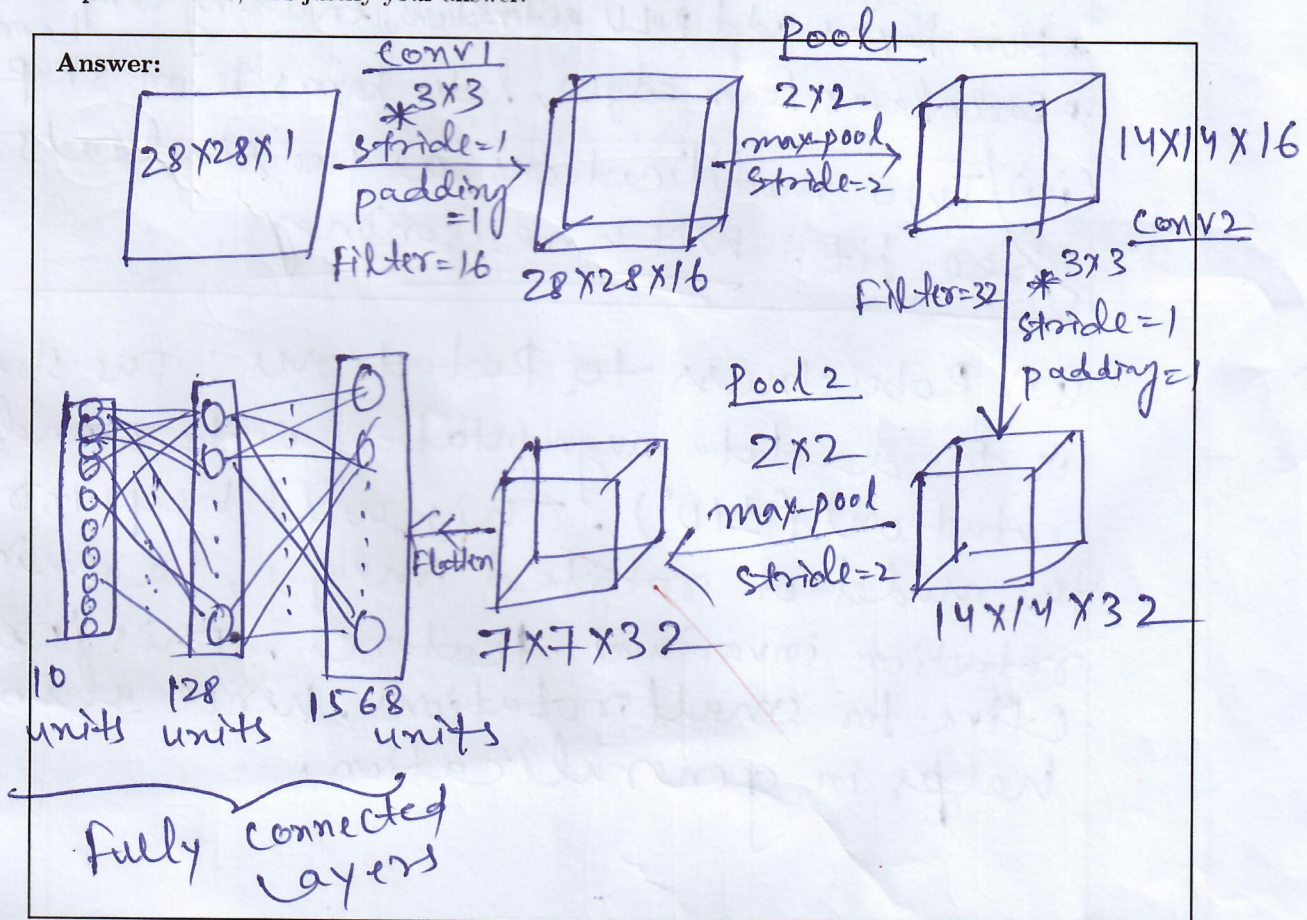
Question 4 continues on the next page...

- (iv) output gate: Selects relevant output features.
- (v) Additive updates: Ensure stable gradient flow, enabling long-term dependency learning.

This makes LSTMs ideal for time-series tasks requiring extended context.

(c) You are designing a Convolutional Neural Network (CNN) model for image classification on a dataset of images, where each input image is of size $28 \times 28 \times 1$ ($W \times H \times C$). The model must be lightweight for deployment on low-power edge devices, while maintaining good classification accuracy. The initial architecture is defined as follows:

- **Convolutional Layer 1:** 16 filters of size 3×3 , stride 1, padding 1
 - **Max Pooling Layer 1:** 2×2 , stride 2
 - **Convolutional Layer 2:** 32 filters of size 3×3 , stride 1, padding 1
 - **Max Pooling Layer 2:** 2×2 , stride 2
 - **Fully Connected Layer:** 128 units
 - **Output Layer:** 10 units (for classification)
- (i) Draw the CNN architecture. Calculate the total number of parameters (including biases) in the given CNN model.
- (ii) Explain the motivation for using small filter sizes (e.g., 3×3) and multiple layers instead of using a single large filter (e.g., 7×7) in a single convolutional layer.
- (iii) Suppose you want to reduce the number of parameters in the model without significantly degrading performance. Suggest two modifications to the model and explain how each affects computation and accuracy.
- (iv) Consider an adversarial scenario where input images are rotated by small angles (e.g., $\pm 10^\circ$). Propose one architectural change or training strategy to improve the model's robustness to such perturbations, and justify your answer.



Answer: Parameter Calculation

$$\text{Conv1: } 3 \times 3 \times 1 \times 16 + 16 = 160, \text{ pool1: } 0$$

$$\text{Conv2: } 3 \times 3 \times 16 \times 32 + 32 = 4640, \text{ pool2: } 0$$

$$\text{FC layers: } 1568 \times 128 + 128 + 128 \times 10 + 10 \\ = 200832 + 1290 = 202122$$

$$\boxed{\text{Total} = 206922}$$

(ii) * Three 3×3 layers have fewer parameters than one 7×7 (e.g. 756 vs 784 for similar settings)
 * More layers add ReLU activations, enabling complex pattern learning.
 * Early layers learn edges, later layers learn shapes.

(iii) Two modifications at the students see fit. With reasoning

(iv) Robustness to Rotations: The strategy is to do data augmentation with random rotations ($\pm 10^\circ$). This will help in training the model on rotated images, learning rotation invariant features. This is effective for small rotations, hence also helps in generalization.

- (d) **Project:** During the theory classes, you were asked to carry out a mini project using deep learning on a topic of your choice. Reflecting on your implementation and results, answer the following:
- (i) Briefly describe the problem you chose and why deep learning was appropriate for it. What were your input features, target outputs, and the type of model you selected?
 - (ii) What is the dataset that you used to train your model? Describe the characteristics of the dataset. Why do you think that this dataset is appropriate for your problem?
 - (iii) Describe your model with an architecture diagram. Justify your choice of architecture (e.g., CNN, RNN, MLP). What considerations did you make while deciding the number of layers, neurons, activation functions, *etc.*?
 - (iv) Did your model overfit during training? If yes, what signs indicated overfitting? List and explain at least two methods you used or could have used to reduce overfitting.
 - (v) If you are given a choice to improve your model further for real-world deployment, what changes would you make? Justify your answer.

Answer:

Your Project Group Number:

Group Member Names:

Project Title:

31 Mark

→ Check for the project topics as shared in the excel sheet.

— Different for different groups.

Answer:

51 Mark

Check for the project of the
company in the next project.
- Different for the company of
the project.

(e) You are given a classification task on a dataset with the following characteristics:

- The dataset contains **1000 training samples**.
- Each sample has **10 input features**, all numeric and normalized.
- There are **3 target classes** (multi-class classification).

You decide to use a **Multi-Layer Perceptron (MLP)** model for this task.

(i) Propose an MLP architecture suitable for this problem. Draw the architecture. Mention:

- The number of layers (including input and output),
- Number of neurons in each hidden layer,
- Activation functions used in each layer.

Justify your design choices briefly.

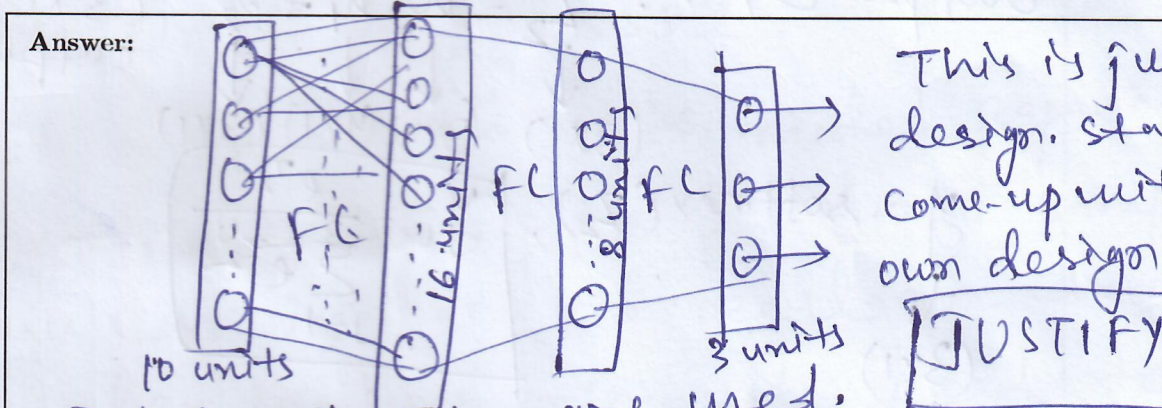
(ii) Consider a single hidden layer MLP with 4 neurons in the hidden layer (using ReLU activation) and 3 output neurons (using softmax activation). Given the input vector: $\mathbf{x} = [0.5, -0.2, 0.1, 0.3, 0.7, -0.1, 0.0, 0.9, 0.6, -0.3]$ Assume the weights and biases are randomly initialised and given as:

- Hidden layer weight matrix W_1
- Hidden layer bias vector $\mathbf{b}_1 = [0.1, 0.2, 0.0, -0.1]$
- Output layer weight matrix W_2
- Output layer bias vector $\mathbf{b}_2 = [0.0, 0.1, -0.2]$

Write down the formulas to compute the final output of the MLP from the input $\mathbf{x}$. Indicate the dimensionality of each step. **You are not required to compute numerical values.**

(iii) Which loss function would you choose for this multi-class classification task and why? Mention one suitable optimization algorithm and justify your choice.

(iv) During training, you observe that the model performs well on the training set, but poorly on the test set. What is this issue called? Mention the reason for its occurrence.



ReLU Activations are used.

Justification - Softmax is used at the classification layer.

(i) Two hidden layers balance capacity and generalization for 1000 samples.

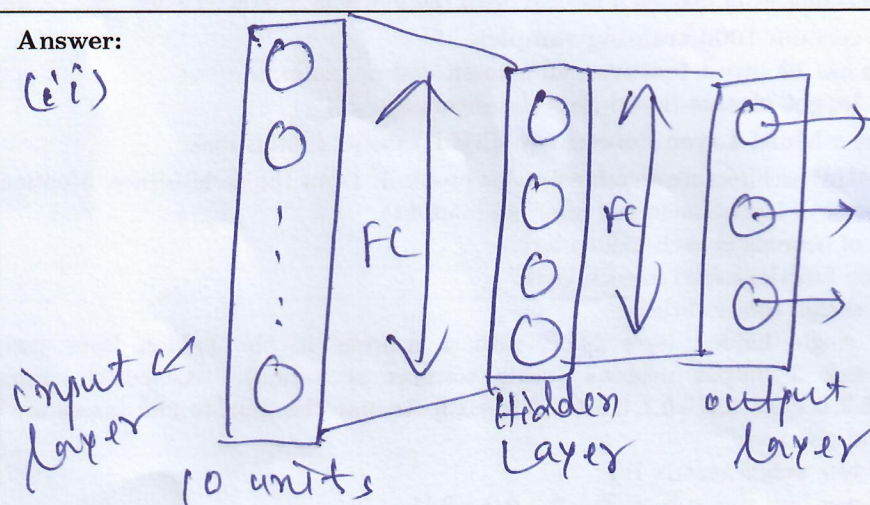
(ii) 16 and 8 neurons in hidden layers create a funnel structure, capturing interactions while reducing parameters.

Question 4 continues on the next page...

(iii) Activations: ReLU for non-linearity, Softmax for 3-class probability output.

Answer:

(i)



Hidden Layer: $z_1 = W_1^T x + b_1$ (4x1) output
 $h_1 = \text{ReLU}(z_1) = \max(0, z_1)$ (4x1) output.

Output Layer: $z_2 = W_2^T h_1 + b_2$ (3x1) output
 $y = \text{softmax}(z_2)$, $y_i = \frac{e^{z_2^{(i)}}}{\sum_{j=1}^3 e^{z_2^{(j)}}}$ (3x1)

(ii) Loss function and Optimizer:

Loss: Categorical Cross Entropy Loss (CCE)

$$L = - \sum_{i=1}^3 y_i \log(\hat{y}_i)$$

Rough Work

Reason: CCE loss is suitable for multi-class classification with softmax activations. It penalizes incorrect class probabilities.

Optimizer: Adam

Reason: Adaptive learning rates, fast convergence, robust for small datasets (1000 samples)

(iv) Overfitting issue.

It occurs with high model capacity, limited training samples (1000 samples in this case), lack of regularization, or complex feature interactions lead to memorizing the training data.

— X —